

COMPLETE LEARNING ROADMAP

Rust

Systems Programming Mastery

From C/C++ history to ownership, lifetimes, async, and the full Rust ecosystem

6 Phases · 40+ Topics · 20 Projects

6 PHASES	40+ TOPICS	20 PROJECTS	~8mo TIMELINE	0 GARBAGE COLLECTOR
-------------	---------------	----------------	------------------	---------------------------

PHASE 0 The Story Before Rust*Why the world needed a memory-safe systems language*

The Problem Rust Was Built to Solve

To understand Rust you must first understand the decades of pain that systems programmers lived through with C and C++. Rust did not emerge from academic theory — it emerged from real production bugs.

A Timeline of Systems Programming

Year	Event
1972	C is created at Bell Labs by Dennis Ritchie <i>Manual memory management: malloc/free. Fast, portable, and dangerous.</i>
1985	C++ adds objects and RAII to C <i>More abstractions, but all of C's memory hazards remain.</i>
1988	The Morris Worm exploits a buffer overflow <i>First major internet worm — caused by C's lack of bounds checking.</i>
1990s	Java and GC languages emerge as 'safe' alternatives <i>Garbage collection solves safety but at the cost of performance and determinism.</i>
2000s	Buffer overflows become the #1 security vulnerability category <i>CVEs pile up in C/C++ codebases. Microsoft estimates 70% of all bugs are memory safety issues.</i>
2006	Graydon Hoare begins Rust as a personal project <i>A Mozilla employee, frustrated by a bug-prone elevator firmware written in C++.</i>
2010	Mozilla sponsors Rust development <i>The goal: a language for writing a next-generation browser engine (Servo).</i>
2015	Rust 1.0 released <i>Stable release with the ownership system finalised. The borrow checker is production-ready.</i>
2016	Rust wins 'Most Loved Language' on Stack Overflow <i>Has won every year since — developers who use Rust love it deeply.</i>
2021	Linux kernel accepts Rust as a second language <i>A seismic shift — Rust joins C as an official kernel language.</i>
2022	Android, Windows, AWS, Google all adopt Rust <i>Major production systems are being rewritten or extended in Rust.</i>
2024	Rust in the Linux kernel ships in production <i>Real Rust code runs in the Linux kernel used by billions of devices.</i>

Why C and C++ Are Dangerous

Buffer Overflow

Writing past the end of an array corrupts adjacent memory. Source of countless CVEs and security breaches.

Null Pointer Dereference

Dereferencing a null pointer crashes the program. Called the 'billion dollar mistake' by its inventor.

Memory Leaks

Forgetting to free allocated memory. The program consumes more RAM over time until it dies or is killed.

Use-After-Free

Accessing memory after it has been freed. Can lead to crashes, data corruption, or remote code execution.

Data Races

Two threads access the same memory concurrently with no synchronisation. Results are undefined and non-deterministic.

Double Free

Freeing the same memory twice. Corrupts the allocator's internal structures and causes undefined behaviour.

Rust's Answer

Rust eliminates all of these bugs at compile time — not by adding a runtime garbage collector, but through its ownership system and borrow checker. If your Rust code compiles, it is guaranteed to be free of memory safety bugs, data races, and use-after-free errors. Zero runtime cost.

PHASE 1 Rust Fundamentals*Variables, types, ownership — the core mental model*

Core Topics — Weeks 1 to 4

Rust has a steeper initial learning curve than most languages because it introduces the ownership system — a fundamentally new way of thinking about memory. These weeks build the foundation.

Variables & Mutability let vs let mut. Rust is immutable by default — you must opt into mutation explicitly.	Primitive Types i8/i16/i32/i64/i128/usize, u8-usize, f32/f64, bool, char. All sized and explicit.
Compound Types Tuples (fixed heterogeneous) and arrays (fixed homogeneous). Both stack-allocated.	Ownership Rules Each value has one owner. Ownership can be moved. When the owner goes out of scope, the value is dropped.
Move Semantics Assigning a non-Copy type transfers ownership. The original variable becomes invalid.	Borrowing & References &T (shared reference) and &mut T (exclusive reference). The borrow checker enforces the rules.
The Borrow Checker You can have many &T OR one &mut T, never both at the same time. This prevents data races.	Slices References to a contiguous sequence: &[T] and &str. No allocation — just a pointer and length.
Functions fn keyword, typed parameters and return types. Expressions vs statements. The last expression is returned.	Control Flow if/else as expressions, loop, while, for...in with iterators. if let and while let patterns.

Phase 1 Projects

☐	CLI Calculator A command-line calculator that parses expressions from stdin. Forces you to handle user input, parse strings into numbers, and manage ownership of string data. <i>Concepts: variables · mutability · functions · control flow · string handling</i>
☐	Stack Implementation Build a generic stack using a Vec<T>. Implement push, pop, peek, and is_empty. This forces you to think about ownership when items are moved in and out. <i>Concepts: ownership · move semantics · generics basics · Option</i>

PHASE 2 Structs, Enums & Pattern Matching*Rust's type system for modelling real-world data*

Core Topics — Weeks 5 to 8

Structs

Named fields (struct Point { x: f64, y: f64 }), tuple structs, and unit structs. The backbone of Rust data modelling.

Enums

Enums whose variants can hold data: enum Shape { Circle(f64), Rect(f64, f64) }. Far more powerful than C-style enums.

Result<T, E>

The error type: Ok(T) or Err(E). Forces you to handle errors explicitly. Replaces exceptions.

if let / while let

Concise pattern matching for a single case without the full match verbosity.

Generics

Generic functions and structs: fn largest<T: PartialOrd>(list: &[T]). Monomorphised at compile time.

Methods & impl Blocks

Attach functions to structs via impl. &self (read), &mut self (mutate), self (consume).

Option<T>

The safe null: Some(T) or None. Forces you to handle the absent case. Replaces null pointers.

Pattern Matching

match expressions that exhaustively cover all enum variants. Destructuring, guards, and binding.

Traits

Shared behaviour across types — Rust's version of interfaces. Define a set of methods a type must implement.

Trait Bounds

Constrain generics: T: Display + Clone. Only types that implement those traits are accepted.

Phase 2 Projects

**Card Game Engine**

Model a deck of cards using enums (Suit, Rank) and structs (Card, Deck, Hand). Deal cards, score hands — rich pattern matching throughout.

Concepts: structs · enums · pattern matching · methods · Option

**Unit Converter Library**

A library crate that converts temperatures, distances, and weights. Uses traits to define a Convertible behaviour and generics to work with any numeric type.

Concepts: traits · generics · impl blocks · structs

**CSV Record Parser**

Parse CSV files into typed structs. Use Result for error handling at every step — file I/O errors, parse errors, missing fields. The ? operator everywhere.

Concepts: Result · Option · pattern matching · file I/O · ? operator

PHASE 3 Ownership Deep Dive*Lifetimes, smart pointers, and interior mutability*

Core Topics — Weeks 9 to 13

This is the phase that separates Rust developers from those who gave up. Lifetimes and smart pointers are the concepts most people struggle with — but once they click, Rust becomes a pleasure.

Lifetime Annotations 'a syntax. Lifetimes describe how long references are valid. The compiler needs help when multiple references are involved.	Lifetime Elision Common lifetime patterns are inferred automatically. Most functions don't need explicit lifetime annotations.
Box<T> Heap allocation of a single value. Use when you need a known size for a recursive type or a large value.	Rc<T> Reference-counted shared ownership for single-threaded code. Multiple owners, freed when the last Rc drops.
Arc<T> Atomic reference-counted shared ownership, safe across threads. The thread-safe version of Rc.	RefCell<T> Interior mutability: allows mutation through a shared reference. Borrow rules enforced at runtime, not compile time.
Cell<T> Interior mutability for Copy types. No runtime cost, no borrowing — just get and set.	Deref & Drop Traits Deref controls what * does on a type. Drop controls what happens when a value goes out of scope.
The Weak<T> Pointer A non-owning reference to break reference cycles in Rc/Arc graphs.	Memory Layout Stack vs heap, size of types at compile time, zero-cost abstractions. How Rust achieves C-level performance.

Phase 3 Projects

☐	Linked List The classic Rust rite of passage. Build a singly-linked list using Box<T> for the next node. Forces you to deeply understand ownership, recursive types, and the Box heap pointer. <i>Concepts: Box<T> · ownership · recursive types · Option · lifetimes</i>
☐	Shared Graph / Tree Build a tree where multiple nodes can reference a parent using Rc<RefCell<T>>. Model a DOM-like structure with parent/child relationships and shared ownership. <i>Concepts: Rc<T> · RefCell<T> · Weak<T> · interior mutability</i>



Thread-Safe Counter

A counter incremented by multiple threads using `Arc<Mutex<T>>`. Then refactor to use atomic types. Demonstrates the thread-safety guarantees enforced by the type system.

Concepts: `Arc<T>` · `Mutex` · `threads` · `atomics`

PHASE 4 Error Handling, Iterators & Collections*Idiomatic Rust for everyday programming***Core Topics — Weeks 14 to 17**

Result & the ? Operator Propagate errors without <code>unwrap()</code> . The <code>?</code> operator short-circuits on <code>Err</code> , returning it from the enclosing function.	Custom Error Types <code>impl std::error::Error</code> for your own error types. The <code>thiserror</code> crate makes this ergonomic.
Error Conversion <code>impl From<OtherError> for MyError</code> enables <code>?</code> to auto-convert error types.	Vec<T> Growable heap-allocated array. The most common collection. <code>push</code> , <code>pop</code> , <code>iter</code> , <code>retain</code> , <code>sort</code> , <code>dedup</code> .
HashMap<K, V> Key-value store. <code>entry</code> API for efficient insert-or-update. Custom hashers for performance.	HashSet<T> Set of unique values. Union, intersection, difference, and membership testing.
String vs &str <code>String</code> is owned heap-allocated, <code>&str</code> is a borrowed slice. Pass <code>&str</code> to functions, return <code>String</code> from them.	Iterators Lazy, composable, zero-cost. <code>map</code> , <code>filter</code> , <code>fold</code> , <code>chain</code> , <code>zip</code> , <code>enumerate</code> , <code>flat_map</code> , <code>take</code> , <code>skip</code> .
Iterator Adaptors Chaining transformations: <code>iter.filter(x ...).map(x ...).collect:::<Vec<_>>()</code> .	Closures Anonymous functions that capture their environment. <code> x x * 2</code> . <code>Fn</code> , <code>FnMut</code> , <code>FnOnce</code> trait bounds.

Phase 4 Projects

☐	Log File Analyser Parse a log file, extract structured records, group by severity, count occurrences, and output summaries. Iterators and closures throughout — no explicit loops. <i>Concepts: iterators · closures · HashMap · String/&str · Result</i>
☐	Word Frequency Counter Count word frequencies in text using <code>HashMap</code> . Sort by frequency. Handle Unicode correctly using Rust's char-aware string methods. <i>Concepts: HashMap · iterators · String · sorting · closures</i>
☐	HTTP Client CLI A command-line tool that makes HTTP requests using the <code>reqwest</code> crate. Full error propagation with custom error types and the <code>?</code> operator throughout. <i>Concepts: Result · custom errors · ? operator · external crates · async basics</i>

PHASE 5 Advanced Rust*Traits deeply, async/await, unsafe, and macros*

Core Topics — Weeks 18 to 26

Trait Objects (dyn) Dynamic dispatch: <code>Box<dyn Trait></code> . Runtime polymorphism when the concrete type isn't known at compile time.	Associated Types type <code>Output</code> in a trait definition. One implementation per type instead of multiple with generics.
Default Trait Implementations Provide default method bodies in trait definitions. Types can override or inherit them.	Supertraits Trait <code>A: B</code> means implementing <code>A</code> also requires implementing <code>B</code> .
async/await Rust's cooperative async model. <code>async fn</code> returns a <code>Future</code> . <code>.await</code> drives it. Tokio is the standard runtime.	Futures & the Runtime Futures are lazy — they do nothing until polled. The async runtime (Tokio/async-std) does the polling.
Concurrency with Threads <code>std::thread::spawn</code> , join handles, channels (mpsc), and the <code>Send/Sync</code> marker traits.	Unsafe Rust The unsafe block. Dereferencing raw pointers, calling unsafe functions, implementing unsafe traits. Know when and why.
Procedural Macros Code that generates code at compile time. <code>#[derive(...)]</code> , attribute macros, function-like macros.	WASM & Embedded Rust compiles to WebAssembly. It also runs on bare-metal microcontrollers with <code>no_std</code> .

Phase 5 Projects

☐	Async Web Scraper An async scraper using Tokio and <code>reqwest</code> that fetches multiple URLs concurrently with <code>tokio::join!</code> or <code>FuturesUnordered</code> . Full error handling with retries. <i>Concepts: async/await · Tokio · futures · concurrent execution · error handling</i>
☐	Thread Pool Build a thread pool from scratch: a fixed number of worker threads that pull tasks from a shared channel. Implement gracefully shutting down all workers. <i>Concepts: threads · channels (mpsc) · Arc<Mutex> · drop semantics · Send</i>
☐	Derive Macro Write a custom <code>#[derive(Describe)]</code> procedural macro that generates a <code>describe()</code> method on any struct, printing its field names and values. Deep compiler internals.

Concepts: proc macros · syn/quote crates · TokenStream · AST traversal



REST API with Axum

A full REST API server using the Axum framework on Tokio. Typed routes, extractors, middleware, state sharing with Arc, and JSON serialisation with serde.

Concepts: async · Axum · serde · Arc<State> · handlers · routing

PHASE 6 Ecosystem, Cargo & Real-World Rust*Tooling, testing, publishing, and the crate ecosystem*

Core Topics — Ongoing

Cargo In Depth

Workspaces, features flags, build scripts (build.rs), cargo profiles, custom commands, cargo-nextest.

Serde

The universal serialisation framework. Derive Serialize/Deserialize on your types for JSON, TOML, YAML, MessagePack, and more.

Tokio

The async runtime. tokio::main, spawn, select!, join!, channels, timers, file I/O, and the Tokio console for debugging.

sqlx / Diesel

Type-safe database access. sqlx checks your SQL queries at compile time against a live database schema.

no_std & Embedded

Rust without the standard library for bare-metal and microcontroller targets. The Embassy async embedded framework.

Testing

#[test] unit tests, #[cfg(test)] modules, integration tests in /tests, doc tests, property-based testing with proptest.

Clap

CLI argument parsing with derive macros. The standard way to build command-line tools in Rust.

Tracing

Structured, levelled logging and diagnostics. The modern replacement for the log crate in async code.

Criterion

Micro-benchmarking framework. Statistical analysis of performance, regression detection.

Publishing Crates

crates.io publishing, semver, documentation with rustdoc and docs.rs, README and license setup.

Capstone Projects

**Key-Value Store (Redis-like)**

A TCP server implementing a subset of the Redis protocol: GET, SET, DEL, EXPIRE. Async with Tokio, concurrent client connections, persistence to disk with serde.

Concepts: async · TCP · Tokio · serde · Arc<Mutex> · protocol design

**CLI Tool — Published to crates.io**

A useful CLI tool (e.g. a file renamer, a diff tool, a password generator). Built with Clap, tested thoroughly, documented, and published to crates.io as a real crate.

Concepts: Clap · testing · rustdoc · cargo publish · error handling

**Interpreter / Language**

Implement a simple expression language or scripting language: lexer, parser, AST, and evaluator. This is the ultimate Rust project — touches every advanced concept.

Concepts: [enums](#) · [Box<T>](#) · [traits](#) · [pattern matching](#) · [recursive types](#) · [testing](#)

Essential Crates to Know

serde / serde_json Serialisation/deserialisation. Used in essentially every Rust project.	tokio The async runtime. The foundation for network and I/O bound async applications.
reqwest HTTP client built on Tokio. The standard way to make HTTP requests.	axum / actix-web Web frameworks. Axum is the modern Tokio-native choice; actix-web is the high-performance veteran.
clap CLI argument parsing. Derive macros make complex CLI interfaces effortless.	thiserror / anyhow thiserror for library error types; anyhow for application-level error handling.
rayon Data parallelism. Turn any iterator into a parallel iterator with <code>.par_iter()</code> .	sqlx / diesel Database. sqlx is async and compile-time SQL verified; diesel is sync and DSL-based.
tracing Structured logging and instrumentation. Works with Tokio's async context.	proptest / quickcheck Property-based testing — generate random inputs and verify invariants.

The Most Important Insight

Rust's learning curve is front-loaded. The first month is hard because you are learning a new mental model — ownership — not just new syntax. Once the borrow checker becomes intuitive, Rust is one of the most enjoyable languages to write. You stop debugging memory bugs entirely and focus on the actual problem.

Rust Learning Roadmap · 6 Phases · ~8 Months